

## HW2-5 作业感受（第二次挑战）

### （1）如何完成第二次挑战

整体上是接着第一次挑战的环境往下做：SGLang 0.5.14 的 CPU 服务在真机 WSL2

上已经跑通，所以这次的重点是把测量做扎实。我先按任务书把硬性参数列成清单（每组 32 条、并发 8、temperature=0、max\_new\_tokens=16、ignore\_eos、sampling\_seed=2026），写进脚本的常量里，避免手滑。测量脚本每次跑之前先 flush \_cache，shared 组额外发一条预热请求把 2048 token 的共享前缀写进 RadixCache。指标全部取服务端返回的 meta\_info (prompt\_tokens、cached\_tokens)，不用客户端自己估。跑完一组就打开 requests.jsonl 抽几行，对一下 cached\_tokens 和 summary.json 的命中率能不能对上。另外考虑到执行顺序可能带来系统性偏差，我把两组的顺序交换后又跑了一轮（run-1 先 shared 后 dispersed，run-2 反过来），两轮结论一致才敢写进报告。

### （2）最困难的部分

第一个坎是编译：这台机器的 CPU 没有 AVX-512 和 AMX，而 sgl-kernel

的 CPU 后端默认按 x86-64-v4 编译，装完一 import 就 Illegal in

struction。一开始完全摸不着头脑，后来用逐库 LD\_PRELOAD 探测加上读 CMake

Lists 才定位到是 -march 标志的问题，把它改成 AVX2 基线重编才通过。第二个坎

是测出来的现象不好解释：纯 CPU 单进程部署下，HTTP 接收和调度计算是串行的，服务端日志显

示每轮 8 条请求里只有 1 条先被准入，其余 7 条要等十几秒才整批进来，TTFT 和 TPO

T 于是变成同一段固定时长在两个窗口之间的分配，单看哪一组都会误读。最后我采取的办法是跨组比较只看端到端时延和吞吐，命中率则用服务端计数器，不受时序影响。CPU 上一组实验要好几分钟，试错成本高，也逼着我每次重跑前把参数检查一遍再发请求。

### （3）对科研的启发

第一，结论要有稳健性意识：同一实验换个顺序再跑一遍，是成本很低的保险。第二，异常读数不一定是错误，可能是系统结构（比如这里的准入串行化）造成的，解释不了就去找服务端的直接证据（日志里的 #queue-req、#cached-token），而不是硬套结论。第三，环境本身也是工作量：在没有 AVX-512 的消费级 CPU 上跑为服务器芯片设计的代码，补丁、降级、重编这些事没人替你做，但正是这些过程让人理解系统每一层在干什么。第四，数据要能溯源，报告里的每个数字我都要求能对回 requests.jsonl 的某一行，这样写结论的时候才有底气。